Intro to Deep Learning

Lecture 05: CNNs From First Principles

Stanislav Don

DS212 - Intro to Deep Learning

Today

By the end of the lecture, you should be able to answer:

1. Why are MLPs a poor fit for raw images?
2. What does a convolutional layer compute?
3. What are channels, filters, and feature maps?
4. How do padding, stride, and pooling change shape?
5. What is a receptive field?
6. How do we build a simple CNN for FashionMNIST?

Lecture rhythm: image structure -> local kernels -> shape bookkeeping -> CNN block

Recap From Lecture 04

Last time:

```
model quality = architecture + optimization + regularization
```

Today we focus on architecture.

For images, the architecture should respect image structure:

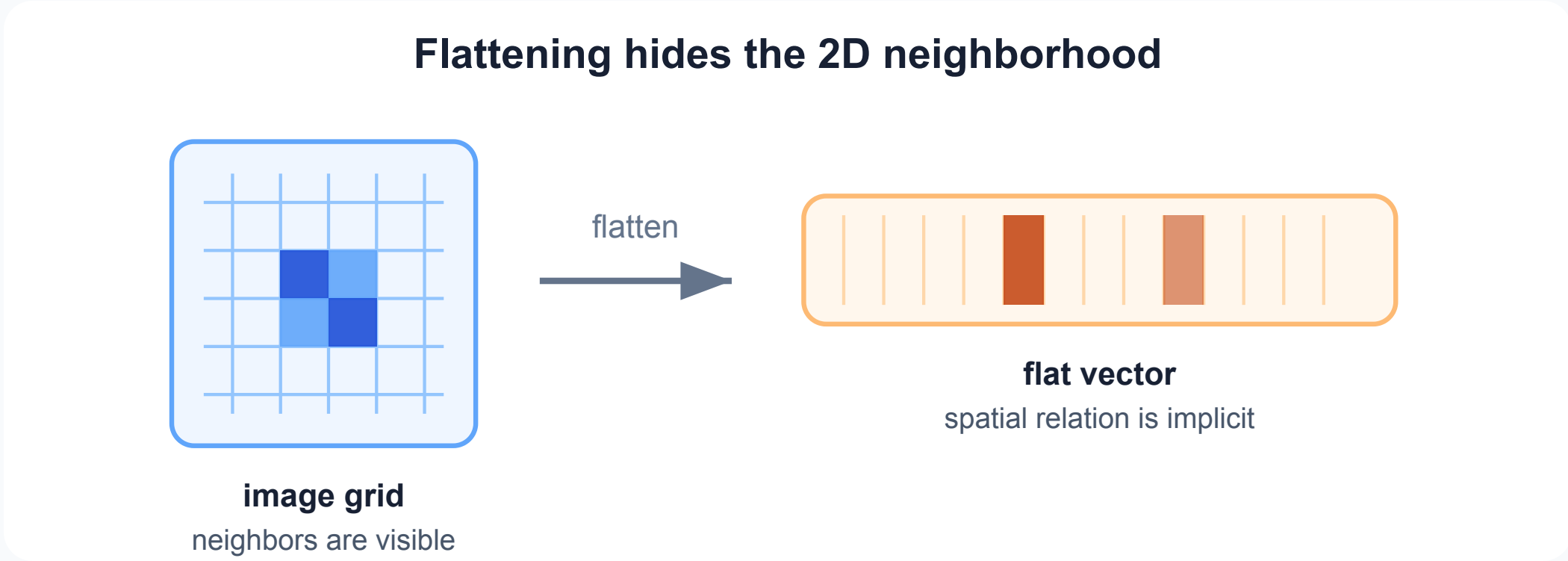
```
nearby pixels are related
patterns can appear in different locations
```

Part 1

Why Images Need Structure

Flattening works, but it ignores what images are

The Problem With Flattening Images



An MLP can still learn, but it must rediscover spatial structure from scratch.

Image Tensor Shape

In PyTorch, images usually have shape:

```
[B, C, H, W]
```

Meaning:

```
B = batch size
C = channels
H = height
W = width
```

For FashionMNIST:

```
[B, 1, 28, 28]
```

Why MLPs Struggle With Images

MLPs can classify images.

But they do not use two important assumptions:

Locality:

```
nearby pixels form useful patterns
```

Translation sharing:

```
the same pattern can matter in many places
```

CNNs build these assumptions into the architecture.

Parameter Count Problem

Suppose the first hidden layer has 512 units.

For a flattened FashionMNIST image:

```
784 inputs * 512 hidden units = 401,408 weights
```

For a small RGB image:

```
32 * 32 * 3 = 3,072 inputs
3,072 * 512 = 1,572,864 weights
```

The first dense layer grows quickly.

Part 2

Convolution

A learned local operation repeated across the image

CNN Idea

Use a small set of weights many times across the image.

Instead of:

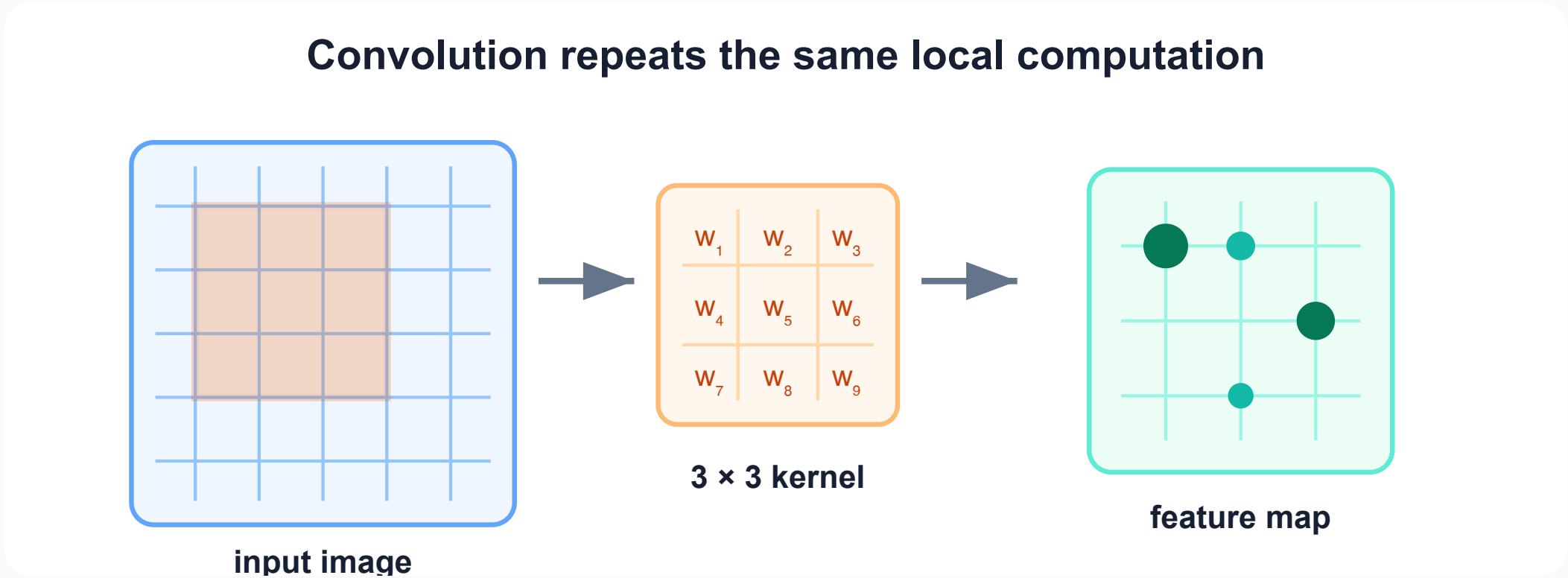
```
every pixel connects to every hidden unit
```

we use:

```
a small kernel looks at a local patch
the same kernel slides across the image
```

This is called weight sharing.

Sliding Kernel



Each local patch produces one output value.

A Kernel

A kernel is a small matrix of learnable weights.

Example:

```
3 x 3 kernel
```

It looks at a local patch:

```
3 x 3 pixels
```

At each location, it produces one number.

Convolution: Local Computation

At one image location:

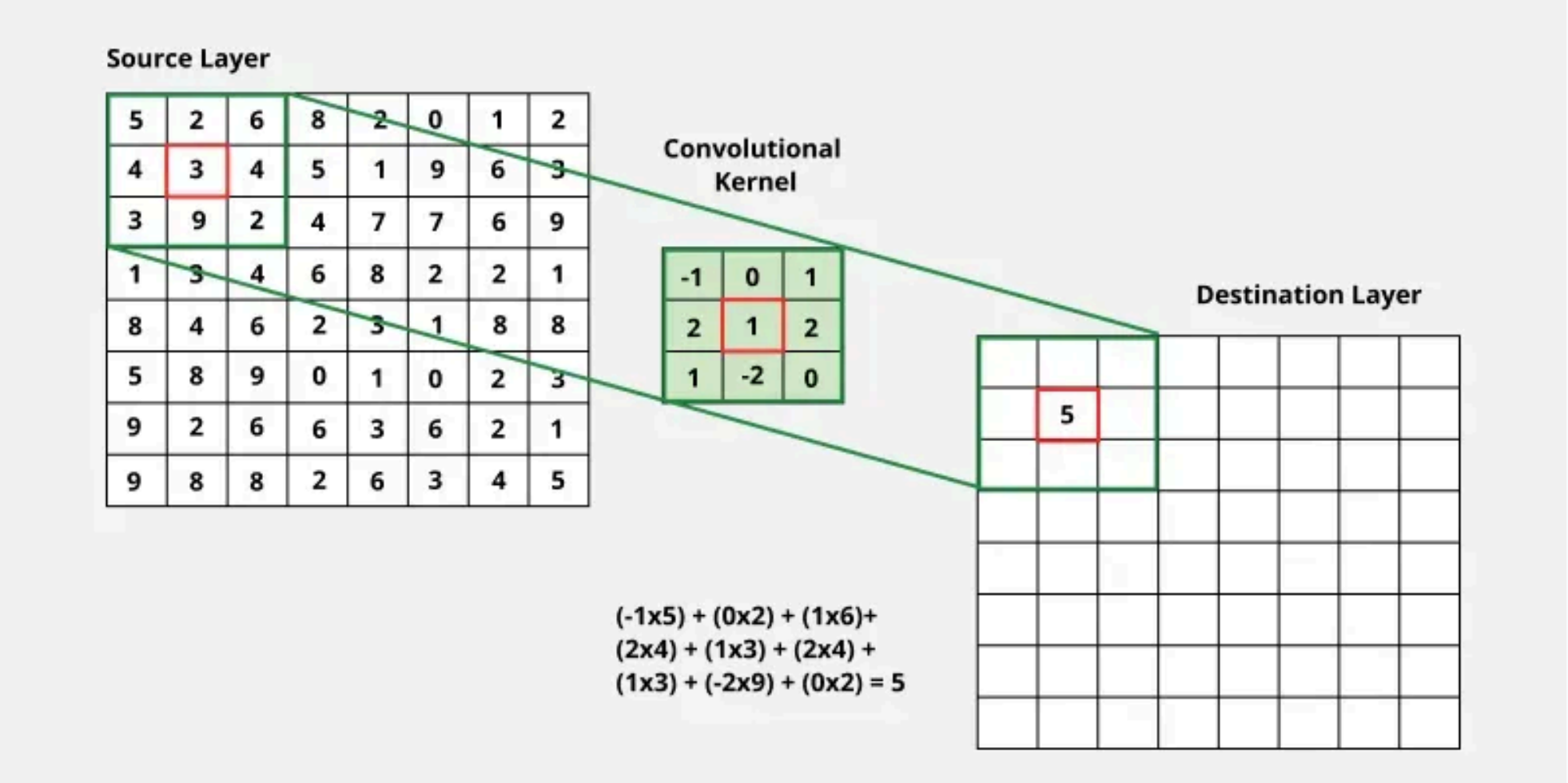
```
patch values * kernel weights
sum the results
add bias
```

Then repeat the same operation at nearby locations.

The output is a new image-like grid:

```
feature map
```

Kernel In Work



The highlighted source patch and kernel weights produce one destination-cell value.

What Does A Kernel Learn?

Early kernels often detect simple local patterns:

- horizontal edges
- vertical edges
- corners
- small textures

In deep learning, we do not manually design these kernels.

They are learned from data by backpropagation.

Feature Map

A feature map answers:

```
where does this learned pattern appear?
```

High value:

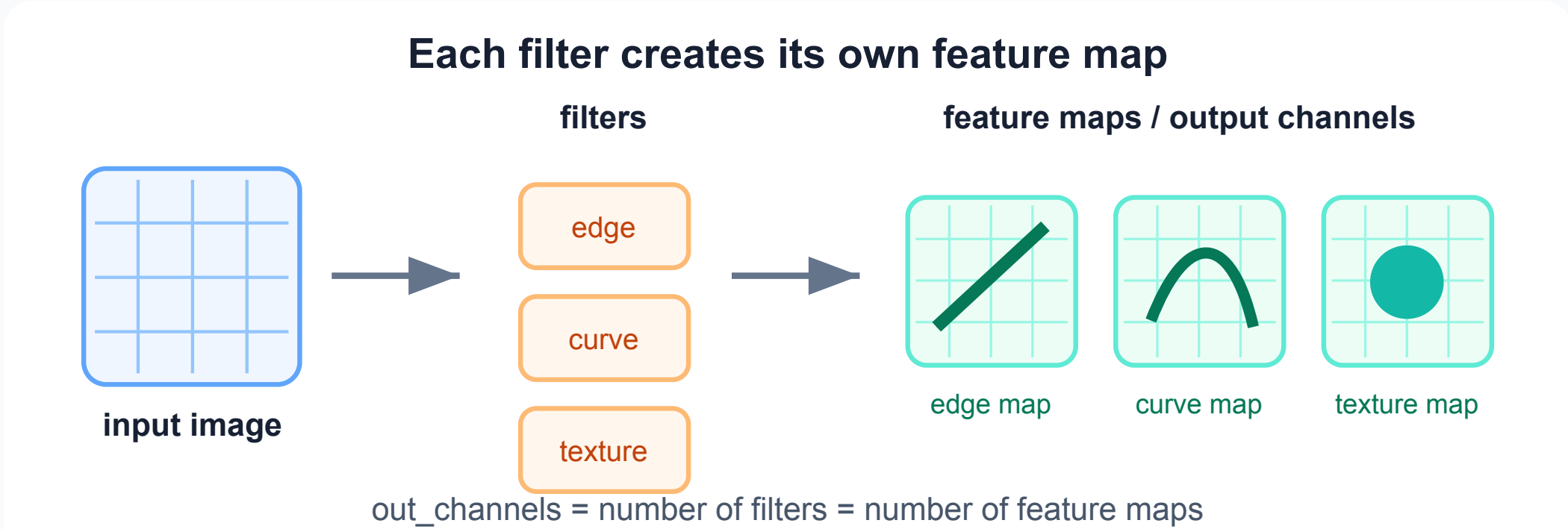
```
pattern is strongly present here
```

Low value:

```
pattern is weak or absent here
```

CNNs turn pixels into maps of learned features.

Filters And Feature Maps



One filter produces one feature map.

Several filters produce several output channels.

Part 3

Channels And Shapes

Conv layers transform stacks of grids

One Filter, One Output Channel

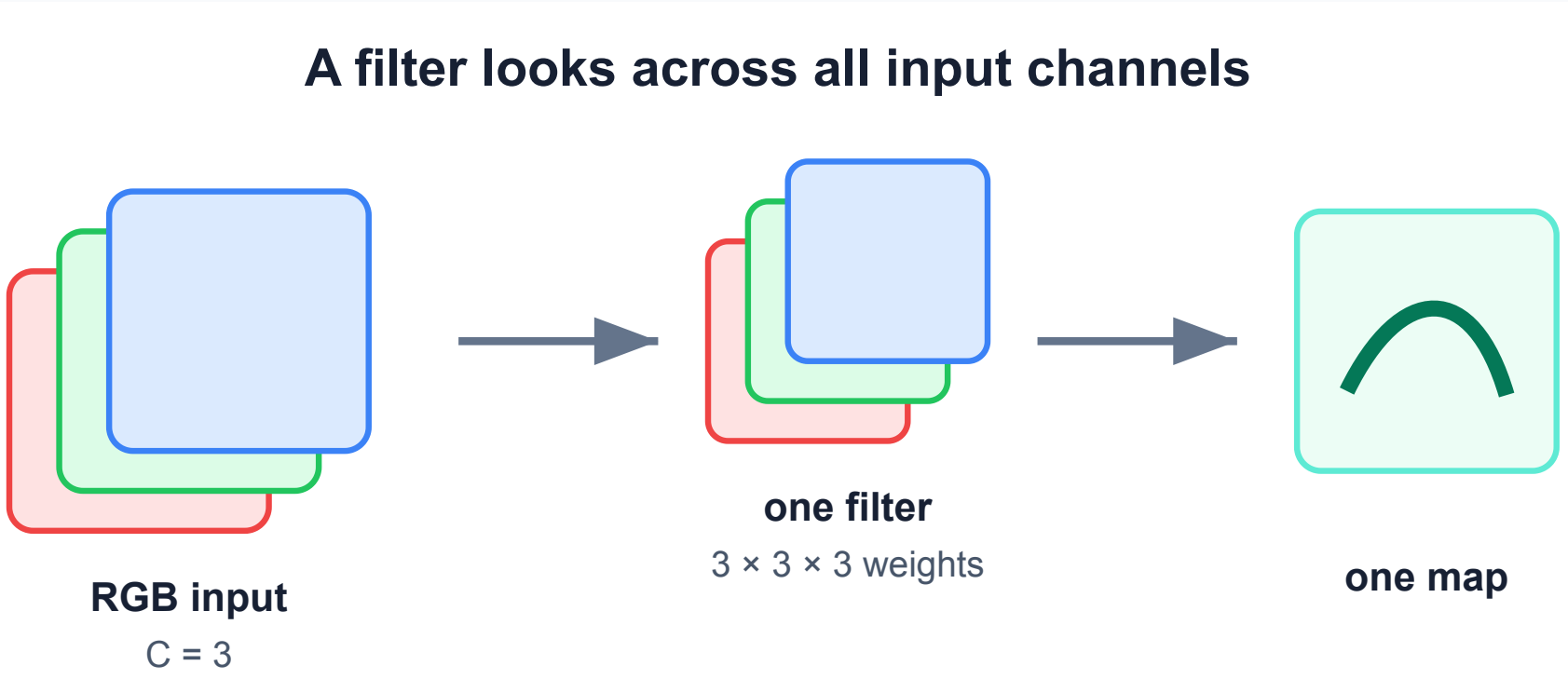
Example:

```
input:  [B, 1, 28, 28]
Conv2d: out_channels = 8
output: [B, 8, 28, 28]
```

The network learns 8 different local pattern detectors.

Each detector creates its own output channel.

Channels



For RGB, one `3 x 3` filter has weights for red, green, and blue.

Conv2d Parameters

PyTorch:

```
nn.Conv2d(
    in_channels=1,
    out_channels=8,
    kernel_size=3,
    padding=1,
)
```

This means:

```
learn 8 filters
each filter sees 1 input channel
each filter is 3 x 3
```

Parameter Count In Conv2d

For a convolution layer:

```
parameters = out_channels * in_channels * kH * kW
```

plus one bias per output channel if bias is used.

Example:

```
Conv2d(1, 8, kernel_size=3)
8 * 1 * 3 * 3 = 72 weights
```

Much smaller than a dense layer on flattened pixels.

Padding And Stride Vocabulary

Before the output-size formula, name the moving parts:

```
kernel size = how large the local window is
padding     = how many border pixels we add
stride      = how far the kernel moves each step
```

Intuition:

- more padding keeps feature maps larger
- larger stride makes feature maps smaller
- kernel size controls the local patch size

Output Size Formula

For one spatial dimension:

$$out = \left\lfloor \frac{in + 2p - k}{s} \right\rfloor + 1$$

where:

```
in = input size
p  = padding
k  = kernel size
s  = stride
```

Use the same formula for height and width.

Example: Preserve Size

FashionMNIST:

```
input height = 28
kernel size  = 3
padding      = 1
stride       = 1
```

Then:

```
out = floor((28 + 2*1 - 3) / 1) + 1
out = 28
```

So `padding=1` preserves size for a `3 x 3` kernel.

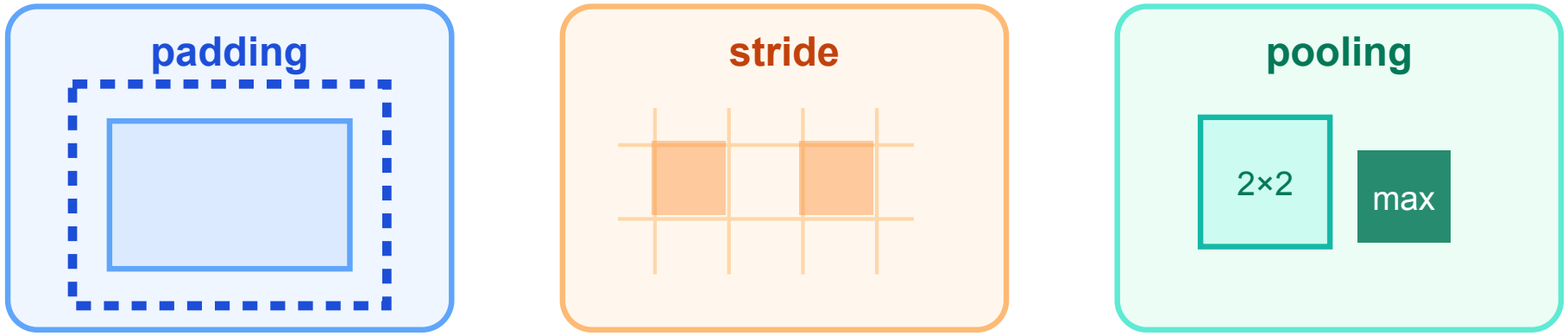
Part 4

Padding, Stride, Pooling

Spatial size is a design choice

Shape Controls

Padding, stride, and pooling control spatial size



adds border values

kernel jumps by s

summarizes neighborhoods

These choices decide how quickly feature maps shrink.

Padding

Padding adds values around the image border.

Why use it?

- keep spatial size after convolution
- let border pixels influence outputs
- control how quickly feature maps shrink

Common pattern:

```
kernel_size = 3
padding = 1
stride = 1
```

This keeps height and width unchanged.

Stride

Stride controls how far the kernel moves.

```
stride = 1: move one pixel at a time
stride = 2: skip every other position
```

Larger stride:

- reduces spatial size
- reduces computation
- loses some spatial detail

Stride is one way to downsample.

Pooling

Pooling summarizes small neighborhoods.

Common example:

```
nn.MaxPool2d(kernel_size=2)
```

This usually means:

```
take max over each 2 x 2 region
halve height and width
```

For FashionMNIST:

```
[B, 8, 28, 28] -> [B, 8, 14, 14]
```

Why Pool?

Pooling gives:

- smaller feature maps
- fewer computations later
- some robustness to tiny shifts

But pooling also removes detail.

Modern CNNs sometimes use strided convolutions instead.

For our first CNN, max pooling is simple and useful.

Part 5

Building A Simple CNN

Conv blocks first, classifier last

Activation Still Matters

A convolution is a linear operation.

Without activation functions:

```
stacked conv layers are still limited
```

So CNN blocks usually contain:

```
Conv2d -> ReLU
```

Often also:

```
Conv2d -> BatchNorm -> ReLU
```

BatchNorm comes in Lecture 06.

A Simple CNN Block

Common beginner block:

```
Conv2d
ReLU
MaxPool2d
```

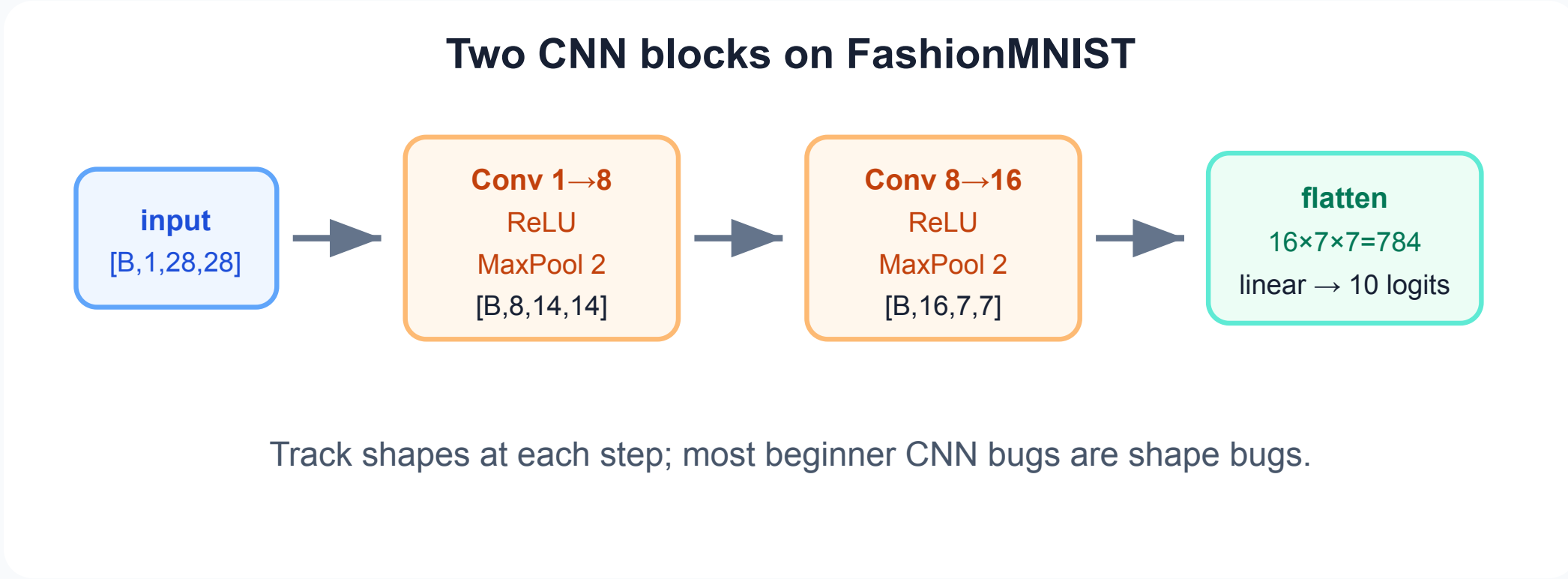
Example:

```
nn.Conv2d(1, 8, kernel_size=3, padding=1)
nn.ReLU()
nn.MaxPool2d(2)
```

Shape:

```
[B, 1, 28, 28] -> [B, 8, 14, 14]
```

Two Blocks On FashionMNIST



After the second pooling layer, the feature tensor is `[B, 16, 7, 7]`.

Flatten Size

Before a linear classifier, flatten everything except batch:

```
[B, 16, 7, 7] -> [B, 16 * 7 * 7]
```

So:

```
16 * 7 * 7 = 784
```

The classifier can start with:

```
nn.Linear(16 * 7 * 7, hidden_dim)
```

Wrong flatten size is a very common CNN bug.

Basic CNN Architecture

For image classification:

```
image
-> convolution blocks
-> flatten or global pooling
-> linear classifier
-> logits
```

For our seminar:

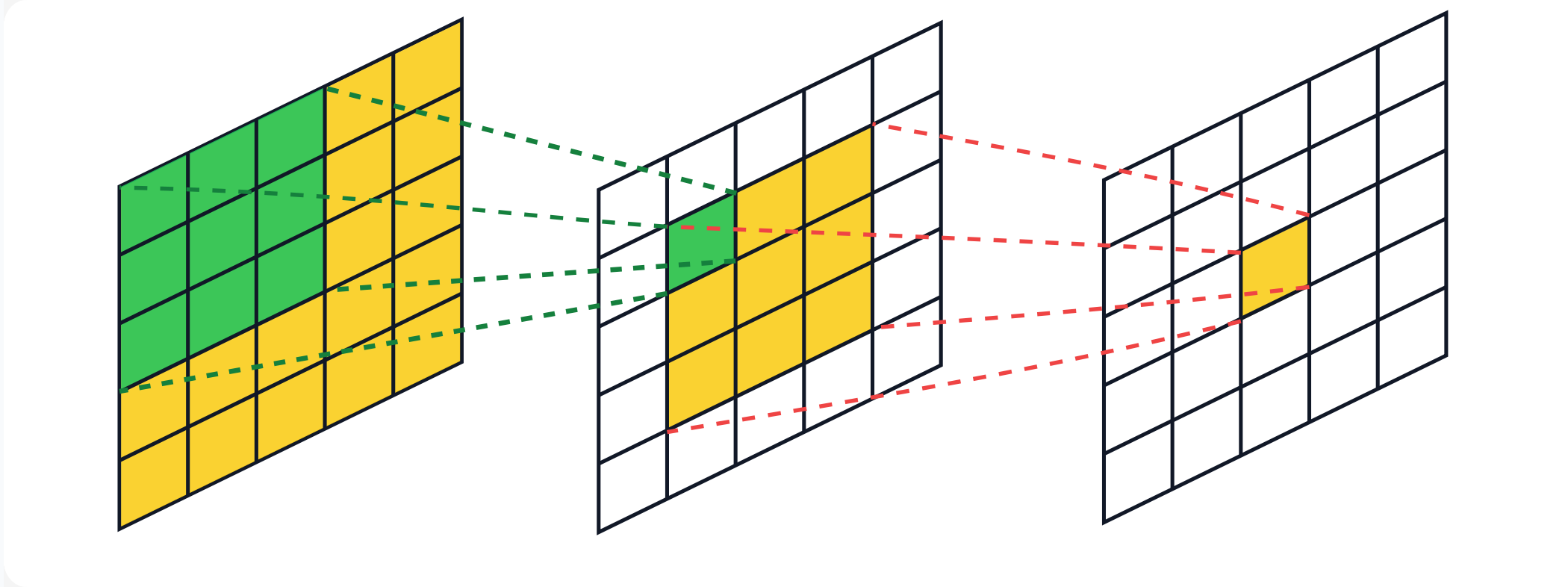
```
FashionMNIST image
-> two conv/pool blocks
-> linear classifier
-> 10 logits
```

Part 6

Why CNNs Work Well For Images

Local patterns become larger patterns

Receptive Field



The receptive field is the input region that can affect one output value.

How Receptive Field Grows

A single `3 x 3` convolution lets one output cell see:

```
3 x 3 input pixels
```

Stack another `3 x 3` convolution:

```
one second-layer cell combines a 3 x 3 neighborhood
of first-layer cells
```

Those first-layer cells looked at overlapping input patches.

So the visible input region grows:

```
1 x 1 -> 3 x 3 -> 5 x 5
```

Receptive Field Calculation

For a stack of layers, track two numbers:

```
r = receptive field size
j = jump between neighboring output cells in input pixels
```

Start:

```
r = 1, j = 1
```

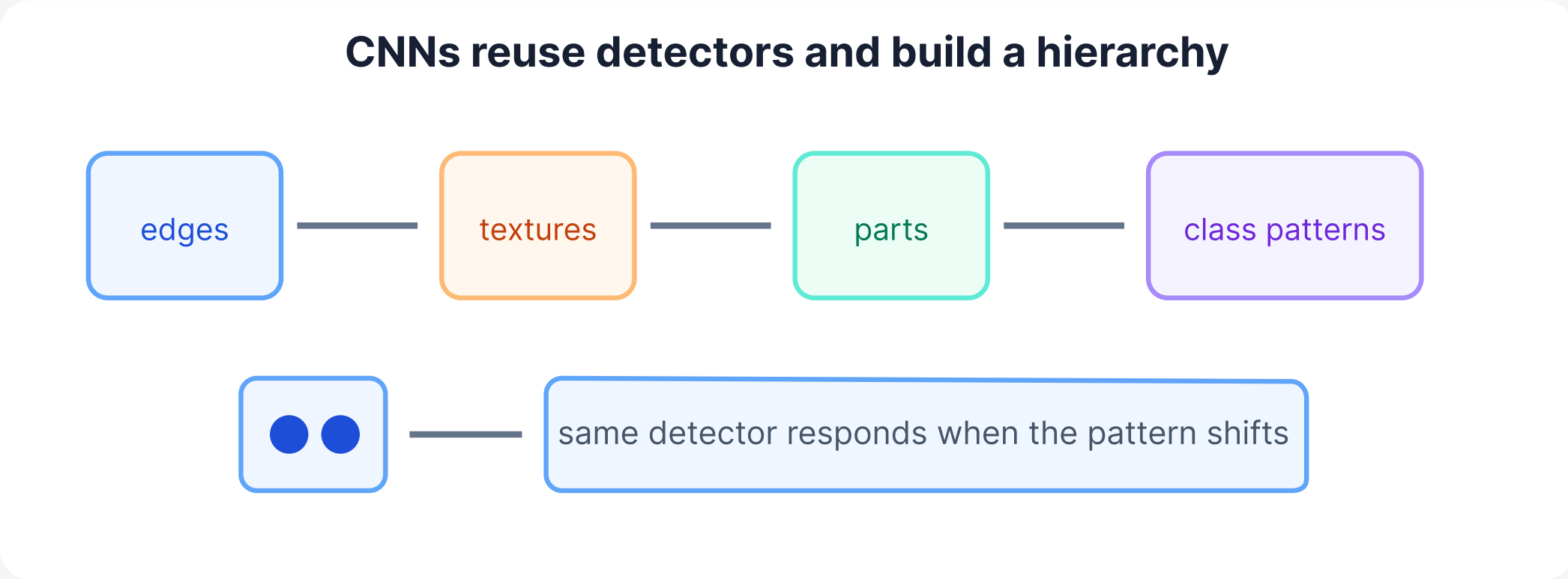
For a layer with kernel size `k` and stride `s`:

$$r_{new} = r + (k - 1)j$$

$$j_{new} = j \cdot s$$

Padding changes border behavior, but not the receptive field size.

Feature Hierarchy And Shifts



Convolution is translation equivariant: when a pattern shifts, the feature response shifts too.

CNN vs MLP

Both can solve FashionMNIST.

But CNNs usually learn better image features because they use:

- local connectivity
- weight sharing
- feature maps
- hierarchical composition

In seminar, compare:

```
accuracy
parameter count
mistakes
training speed
```

What We Are Not Deriving Today

We are not deriving convolution backpropagation.

Important practical idea:

```
Conv2d has learnable weights
autograd computes gradients
optimizer updates kernels
```

You already know the training loop:

```
forward -> loss -> backward -> step
```

CNNs use the same learning mechanism.

Part 7

Seminar Bridge

Track tensor shapes at every stage

Common Shape Mistakes

Mistake 1:

```
using [B, H, W, C] instead of [B, C, H, W]
```

Mistake 2:

```
wrong input channels
```

Mistake 3:

```
wrong flatten size
```

When confused, print shapes inside `forward()`.

Mini-Check: Shape Flow

Input:

```
x: [B, 1, 27, 27]
```

Network:

```
Conv2d(1, 8, kernel_size=3, padding=1)
ReLU()
MaxPool2d(kernel_size=3)
Conv2d(8, 16, kernel_size=3, padding=1)
ReLU()
MaxPool2d(kernel_size=3)
```

What is the final tensor shape?

Answer

Convolutions with `3 x 3`, `padding=1`, `stride=1` preserve height and width.

ReLU does not change shape.

Pooling with `kernel_size=3` uses stride `3` by default:

```
[B, 1, 27, 27]
-> [B, 8, 27, 27]
-> [B, 8, 9, 9]
-> [B, 16, 9, 9]
-> [B, 16, 3, 3]
```

Mini-Check: Receptive Field

Suppose we stack two convolutions:

```
Conv 3 x 3, stride 1, padding 1
Conv 3 x 3, stride 1, padding 1
```

One output cell in the second layer can depend on how large a patch of the original image?

```
3 x 3?   5 x 5?   6 x 6?
```

Answer

It can depend on a **5 x 5** patch.

Using the recurrence:

```
start: r = 1, j = 1
after conv 1: r = 1 + (3 − 1)∗1 = 3
after conv 2: r = 3 + (3 − 1)∗1 = 5
```

Padding preserves spatial size, but it does not make the receptive field smaller.

Seminar Bridge

In Seminar 05, you will:

1. inspect FashionMNIST batches
2. print shapes after `Conv2d` and `MaxPool2d`
3. build a two-block CNN
4. train it with `CrossEntropyLoss`
5. compare it to an MLP
6. inspect misclassified images

Main debugging habit:

```
track tensor shapes at every stage
```

Mini-Checklist

Before training a CNN, check:

- input shape is `[B, C, H, W]`
- `in_channels` matches the data
- output logits shape is `[B, num_classes]`
- target shape is `[B]`
- loss is `CrossEntropyLoss`
- flatten size is correct

Most first CNN bugs are shape bugs.

Takeaways

CNNs are designed for image structure.

Key ideas:

- kernels learn local patterns
- filters create feature maps
- channels store multiple learned features
- padding, stride, and pooling control shape
- deeper layers see larger receptive fields

Next lecture:

```
how to train deeper CNNs well
```